

SynergyLabs Assignment

Implementation explanation, scenarios, and technology rationale

Problem 1 Cost-Efficient RAG

Problem 2 LLM-as-Judge

What this document covers

- The business problem each component solves and the end-to-end execution scenario.
- Why the chosen Python, sqlite-vec, FastAPI, structured-schema, and evaluation tools fit the assignment.
- The generated retrieval, cost, bias, and validation evidence included with the project.

Offline-first demonstration

The submitted sample runs without an API key. OpenAI is optional and configured only through environment variables.

Honest scope

The included metrics come from a small fixed test corpus. Cost at 100K to 10M vectors is an explicit planning model, not a vendor quote.

Cost-Efficient RAG Application

Scenario: a support team needs trustworthy answers across product, security, release, SLA, retention, and integration documents without paying for an always-on managed vector database.

PDF / HTML / MD	Extract + chunk	SHA-256 dedupe	384d embedding	sqlite-vec search	Cited answer
-----------------	-----------------	----------------	----------------	-------------------	--------------

How the scenario runs

- An administrator sends local files to **POST /ingest**. The pipeline extracts text, splits it into configurable 450-character chunks with 60-character overlap, assigns metadata, and hashes every chunk.
- The same document can be ingested again safely. The chunk hash is the SQLite primary key, so an existing vector is skipped instead of duplicated.
- A user asks, for example, *What encryption protects customer data at rest?*. The service embeds the query, searches sqlite-vec, applies an optional metadata filter such as *category=security*, and returns a citation-backed answer.
- If the best context is below the configured relevance threshold, the API returns the explicit no-context response rather than inventing an answer.

Why this architecture

Low idle cost

sqlite-vec runs inside a local SQLite file. There is no separate server or permanently allocated vector pod while the app is stopped.

Grounded answers

The answer prompt is restricted to retrieved text and requires document/chunk citations. This makes evidence inspectable by the caller.

Practical retrieval

The store uses vector candidate selection plus a lexical re-ranker, reducing the chance that a small hashing-vector collision outranks direct evidence.

Traceable operations

Each query logs retrieval and generation latency, retrieved chunk count, tokens, provider, and filter to JSON Lines.

Evidence and technology rationale

The included 18-question benchmark covers factual questions from a fixed sample corpus. Retrieval and answer measurements are generated by the supplied evaluation harness.

100% Recall@3	0.944 MRR	0.959 nDCG@3	4.1 ms Retrieval p50
-------------------------	---------------------	------------------------	--------------------------------

Evaluation highlights

- Idempotence was observed: first ingest inserted 11 chunks; second ingest inserted 0 and skipped 11 existing chunks.
- Answer evidence: 100% citation/lexical faithfulness, 78% exact-match containment, and 0.612 lexical answer relevance.
- Latency: p50 4.072 ms and p95 4.816 ms on the bundled local corpus. These are local-demo values, not a distributed production benchmark.

Cost model

Vectors	Embedded sqlite-vec	Managed DB assumption	Estimated savings
100K	\$25.02 / month	\$70 / month	64.3%
1M	\$25.20 / month	\$280 / month	91.0%
10M	\$26.96 / month	\$1,200 / month	97.8%

Assumptions: 384 float32 dimensions plus 512 bytes metadata/index allowance, 50K queries/month, a \$25 shared application VM, disk/backup storage, and illustrative always-on managed pricing. Embedding/generation API spend is excluded because it is store-independent.

Stack choices

- **Python + FastAPI + Uvicorn:** quick HTTP delivery, typed request validation, and easy local development.
- **sqlite-vec + SQLite:** embedded persistence, metadata indexes, deterministic upserts, and no idle control plane. Move to Qdrant/pgvector/managed service for HA, multi-region, high concurrency, or high-scale throughput.
- **Hashing 384d embeddings:** no download or API key for a reproducible demo. The provider boundary supports an OpenAI model when semantic quality is the priority.
- **pypdf + BeautifulSoup:** lightweight support for PDF, HTML, Markdown, and text without a complex document platform.

LLM-as-Judge Evaluation Pipeline

Scenario: a team wants to decide whether a new prompt or model configuration is better, but cannot trust an unexamined judge because judges can favor first position, verbosity, polished confidence, or their own model family.

JSON / YAML suite	Structured rubric	Judge A vs B	Swap B vs A	Aggregate	Audit + validation
-------------------	-------------------	--------------	-------------	-----------	--------------------

How the scenario runs

- A test case contains the input, system prompt, expected answer, and outputs from configuration A and B. The judge produces a strict JSON verdict, then Pydantic validates it. Fenced or embedded JSON is repaired/extracted when possible.
- Both outputs receive pointwise scores for correctness, faithfulness, completeness, instruction following, tone, and safety. The weighted criteria make the reasoning visible instead of relying on a single opaque number.
- For every comparison, the pipeline judges A vs B and then B vs A. The reverse result is mapped back to original labels. If the winner changes, the case is conservatively declared a tie.
- Every judge prompt, raw response, parsed result, token estimate, latency, and configuration is appended to an audit log for replay and investigation.

Bias safeguards implemented

Position bias

Order-swapped pairs are required. The report includes the per-case consistency flag and flip rate.

Verbosity and style

Score anchors and explicit tone rules penalize unsupported padding. Terse-but-correct answers are not penalized merely for being short.

Sycophancy and confidence

Per-criterion grounding against a reference prevents confident or polished phrasing from masking incorrect claims.

Self-enhancement and clustering

Generator families are configured separately from the judge, and 1/3/5 calibration anchors encourage use of the full scale.

Validation evidence and technology rationale

The demonstration suite compares two configurations over eight reference-based factual and instruction-following cases, plus adversarial probes.

87.5% Config B pass rate	8 - 0 B wins - A wins	0% Position flip rate	100% Probe accuracy
------------------------------------	---------------------------------	---------------------------------	-------------------------------

What the report says

- Configuration B is the declared winner. Its mean overall score is 4.888 versus 2.562 for A, with 87.5% versus 12.5% pass rate.
- The order swap check produced 0% flips. Human/gold validation achieved quadratic weighted Cohen's kappa of 0.8567 and pairwise winner agreement of 100%.
- The adversarial suite achieved 100%: it rejected verbose/confidently wrong answers, accepted terse correct answers, and limited the padded-correct answer's score.
- The default offline judge is deliberately deterministic. Its 100% test-retest result proves repeatable pipeline behavior, not real-world LLM reliability. A production release gate must use a distinct external judge model, more human labels, and human review for high-impact cases.

Stack choices

- **Pydantic**: validates structured verdicts and keeps malformed model output from silently corrupting reports.
- **JSON + YAML**: human-readable suites/rubrics with no database required for a take-home demo; JSON Lines audit records make every call append-only and inspectable.
- **Provider abstraction**: offline scoring makes the project runnable without credentials; an OpenAI judge can be selected with environment variables, independently from generator A/B.
- **Custom IR and agreement metrics**: transparent Recall/MRR/nDCG, pass rates, flip rate, adversarial accuracy, and quadratic weighted kappa avoid hiding evaluation logic behind a black box.

Final takeaway

The submission prioritizes cost visibility, grounded answers, reproducible evaluations, and auditability. sqlite-vec is a credible low-idle-cost choice for small or lightly queried corpora; the judge pipeline is a useful regression signal only after its bias checks and human agreement are monitored in the environment where it will be used.